Desenvolvimento para dispositivos móveis

UNIDADE 03

Indo além com Kotlin: programando com interfaces imperativas

Olá, tudo bem? Em nossa última aula trabalhamos com a estrutura de um projeto Android. Agora podemos avançar mais. Kotlin é uma linguagem orientada a objeto, e esse é um conceito fundamental na criação de aplicativos. Assim sendo, vamos avançar nos conceitos da linguagem, **criando aplicações um pouco mais complexas fazendo uso de interfaces imperativas.**



©Melisa/Adobe Stock

É importante destacar que, trabalhar com alguns componentes simples na criação de aplicativos é base para conhecer e usar todos os demais componentes, pois o uso é análogo. Assim, o que vai mudar muitas vezes de projeto para projeto é seu tamanho, e não sua complexidade.

Vamos começar?

Videoaula: Criando um aplicativo de votação com LinearLayout e View Binding

Nesta aula, vamos criar um projeto fazendo uso de componentes simples de interface gráfica imperativa. Estes componentes são ditos simples em função do seu uso direto a partir de seus métodos e propriedades. Fazer uso de alguns destes componentes vai permitir estender os conceitos para tantos outros, compondo assim projetos cada vez mais complexos.

Ao mesmo tempo que veremos novos componentes de interface, vamos usar os novos conceitos vistos nesta semana referentes à linguagem Kotlin. Tudo em Kotlin é feito a partir da orientação a objetos, e isso vai ficar cada vez mais claro de acordo com cada novo projeto criado.



DICA

Para acompanhar o professor durante a explicação e exercitar o projeto, disponibilizamos, a seguir, o arquivo zipado dos códigos.

Aula03_Codigos_Videoaula_.zip

Kotlin: o que mais ele tem a nos oferecer?

Na semana anterior foram trabalhados os conceitos básicos de Kotlin: criação de variáveis e fluxo de controle. Estes são conceitos fundamentais de toda linguagem de programação, independente do paradigma ao qual pertençam. Agora vamos nos aprofundar nos demais aspectos da linguagem, mostrando como Kotlin se apresenta de forma elegante e funcional.

Nullable

Em muitas linguagens de programação, um dos erros mais comuns é tentar usar uma variável que não possui valor, o que gera um erro de referência nula (*NullPointerException*). Em Kotlin temos um conceito que visa a reduzir esse tipo de erro: ***nullable***.

Nullable permite um controle sobre valores nulos. Por padrão, nenhuma variável em Kotlin pode ser nula, a menos que isso seja indicado explicitamente pelo programador. Em outras palavras, o programador autoriza uma variável a receber valores nulos.

A declaração permite a uma variável receber um valor nulo é feita usando o operador (operador de chamada segura – *safe call operator*)?

```
</>
1 | var nome: String? = null
```

No caso anterior, se quisermos saber qual a quantidade de caracteres a variável ***nome*** possui, podemos interrogar seu conteúdo chamando a propriedade ou função desejada (no caso, ***length***).

```
</>
1 | println(nome?.length)  // Usa o operador seguro ?. para evitar err
```

Kotlin exige que uma variável que possa ser nula seja interrogada para acessar suas funcionalidades, evitando que haja um erro caso não apresente um valor inicializado. Se a variável ***nome*** for nula, de forma segura a propriedade ***length*** não será chamada. Caso contrário, ***length*** retornará a quantidade de caracteres.

> *Operador de asserção não nula (!!)*

Embora Kotlin tenha essa ferramenta de segurança à disposição do programador, ele dá liberdade para execução de operações não seguras mesmo com o uso de ***nullable***, algo que linguagens como ***C / C++*** também fazem. Por mais que uma variável possa ter um valor nulo, o operador ***!!*** força a execução de uma chamada sobre essa variável, mesmo que isto resulte em erro.

```
</>
1 | var nome: String? = "Maicris"
2 |
3 | // A variável é nullable, mas temos certeza de que não é nula nest
4 | val tamanho = nome!!.length
5 |
6 | println("O nome tem $tamanho caracteres.")
```

Nesse caso, independente do conteúdo da variável nome estar definido ou não, a chamada à variável será forçada. Se seu valor for ***null***, a execução resultará em um erro: ***Exception in thread "main" kotlin.KotlinNullPointerException***.

Elvis Operator (?:)

O operador ***?:***, chamado ***Elvis Operator***, permite designar um valor padrão para uma execução caso o valor de uma variável seja nulo. Assim, sempre haverá uma saída consistente na chamada.

```
</>
1 | val tamanho = nome?.length ?: 0
2 | println("Tamanho do nome: $tamanho")
```

Nesse caso, se a variável ***nome*** for ***null***, a mensagem exibirá tamanho 0 para o nome.

Função let

A função *let* permite a um bloco de instruções ser executado caso um valor avaliado não seja nulo.

```
</>
1 | var nome: String? = "Maicris"
2 |
3 | nome?.let {
4 |     println("O nome possui ${it.length} caracteres.")
5 | }
```

Nesse caso, a instrução da linha 4 só será executada caso o valor de ***nome*** não seja ***null***. Essa instrução funciona muito bem em conjunto com o operador ***?:***, atuando como se fosse uma condicional ***if*** no código:

```
</>
```

```kotlin
var nome: String? = "Maicris"

nome?.let {
    println("O nome possui ${it.length} caracteres.")
} ?: run {
    println("O valor de nome não está definido.")
}
```

Aqui ambos os caminhos são abordados.

Funções

As funções são blocos de código criados para realizar tarefas específicas. Elas ajudam a organizar, reutilizar e estruturar o programa de forma mais clara.

A sintaxe da chamada de uma função em Kotlin começa pela expressão ***fun***, seguida do nome da função, seus parâmetros e tipo de retorno, como segue:

</>

```kotlin
fun nomeDaFuncao(parametros): TipoDeRetorno {
    // corpo da função
    return valor
}
```

Como exemplo, vamos criar uma função que efetua a soma de dois números e retorna o resultado:

</>

```kotlin
fun somar(a: Int, b: Int): Int {
    return a + b
}

val resultado = somar(5, 3)
println("Resultado: $resultado")
// Resultado:
```

Como em outras linguagens de programação, em Kotlin não é necessário que uma função retorne um valor. Para tanto, basta omitir o tipo de retorno ou usar o tipo ***Unit***.

Parâmetros nomeados

Os parâmetros nomeados (ou *named arguments*) são um dos recursos mais elegantes e legíveis do Kotlin — e fazem uma diferença enorme quando uma função tem muitos parâmetros ou valores padrão.

Por exemplo:

```
</>
1  fun exibirMensagem(nome: String, idade: Int, cidade: String) {
2      println("$nome tem $idade anos e mora em $cidade.")
3  }
4
5  exibirMensagem(nome = "Maicris", cidade = "Curitiba", idade = 30)
```

É importante ressaltar que com essa abordagem, inclusive, a ordem dos parâmetros da função não precisa ser respeitada, necessariamente.

Funções com retorno simplificado

Funções em Kotlin podem apresentar uma forma simplificada, quando forem simples o suficiente para isto, permitindo um código mais limpo e mais legível.

```
</>
1  fun dobrar(x: Int) = x * 2
2
3  fun main() {
4      println(dobrar(6))  // imprime 12
5  }
```

Na função declarada na linha ***1*** é possível perceber o retorno em forma de atribuição. Não há necessidade de declarar tipo de dado, uma vez que a inferência ocorre como em declaração de variáveis.

Orientação a Objetos

A programação orientada a objetos (POO) é um modo de organizar o código que facilita a criação, o entendimento e a manutenção de programas. Em Kotlin, ela é usada para modelar o mundo real por meio de classes e objetos. Uma classe é como um molde que define as características (propriedades) e os comportamentos (funções) de algo. Um objeto é uma instância dessa classe — ou seja, uma versão concreta dela em execução.

</>

```
1  class Pessoa(nomeInicial: String, idadeInicial: Int) {
2
3      // Propriedades declaradas no corpo da classe
4      val nome: String
5      var idade: Int
6
7      // Bloco init (executado quando o objeto é criado)
8      init {
9          nome = nomeInicial
10         idade = idadeInicial
11     }
12
13     // Método da classe
14     fun apresentar() {
15         println("Olá, meu nome é $nome e tenho $idade anos.")
16     }
17 }
18
```

Como pode ser visto anteriormente, uma classe em Kotlin já apresenta seu construtor (dito construtor primário) em sua declaração. Diferente de outras linguagens, em que o construtor primário é declarado no corpo da classe e a partir daí são executadas instruções iniciais com estes dados, Kotlin recebe os parâmetros de inicialização da classe na declaração e pode conter um ou mais blocos ***init***, que são blocos de inicialização.

As propriedades das classes são declaradas em seu corpo na forma de variáveis (com ***val*** ou ***var***), recendo automaticamente métodos ***get*** e ***set***, implícitos, os quais podem ser sobrecarregados para mudar sua forma padrão de execução.

Os métodos são declarados, como em outras linguagens, fazendo uso de todos os conceitos de funções.

As propriedades de uma classe podem também ser declaradas diretamente em seu construtor primário, simplificando o código. Para tanto, bastante colocar os qualificadores ***val*** e ***var*** a frente do nome dos parâmetros.

</>

```
class Pessoa(val nome: String, var idade: Int) {
    fun apresentar() {
        println("Olá, meu nome é $nome e tenho $idade anos.")
    }
}
```

Nesse caso a simplificação do código é grande, uma vez que o bloco de inicialização ***init*** estava sendo usado exclusivamente para atribuir os valores dos parâmetros de construção para as propriedades.

Construtores secundários

Como visto anteriormente, o construtor primário em Kotlin é executado diretamente na passagem de parâmetros da classe.

Ainda podemos criar construtores secundários, que recebam outros parâmetros. Entretanto, a regra primordial é: o construtor secundário sempre deve chamar e atender aos parâmetros do construtor primário. Vamos a um exemplo:

</>

```
1  class Pessoa(nomeInicial: String, idadeInicial: Int) {
2
3      val nome: String
4      var idade: Int
5
6      // Bloco init - executado sempre que o objeto é criado
7      init {
8          nome = nomeInicial
9          idade = idadeInicial
10     }
11
12     // Construtor secundário
13     constructor(nomeInicial: String) : this(nomeInicial, 0) {
14         println("Idade não informada, definida como 0.")
15     }
16
17     fun apresentar() {
18         println("Olá, meu nome é $nome e tenho $idade anos.")
```

Como pode ser visto na linha ***13***, temos o construtor secundário que recebe apenas o nome da pessoa. Porém, para manter a integridade do construtor primário, ele o chama usando a cláusula ***this***, passando o nome recebido e um valor padrão para a idade. Adicionalmente, o construtor secundário pode executar um bloco de instruções, como ocorre com construtores em outras linguagens.

> *Herança*

A herança é um dos pilares fundamentais da POO. Ela permite a uma classe reaproveitar e estender o comportamento de outra, facilitando a reutilização de código e a criação de hierarquias lógicas entre objetos.

Em Kotlin, para uma classe poder ser herdade ou seus métodos sobrescritos, é necessário que ambos sejam declarados com ***open***, como uma forma de proteção. Segue um exemplo:

```
</>
```

```
open class Animal(val nome: String) {
    fun dormir() {
        println("$nome está dormindo.")
    }

    open fun emitirSom() {
        println("O animal faz um som genérico.")
    }
}
```

Nesse exemplo, a classe Animal pode ser herdade, uma vez que foi declarada com ***open***. O mesmo ocorre com o seu método ***emitirSom***, que é específico para cada animal que será criado herdando dessa classe. Já o método ***dormir*** não pode ser sobrescrito, uma vez que todos os animais (em tese) dormem.

</>

```
class Cachorro(nome: String) : Animal(nome) {
    override fun emitirSom() {
        println("$nome está latindo!")
    }
}

fun main() {
    val rex = Cachorro("Rex")
    rex.dormir()         // Herdado da classe Animal
    rex.emitirSom()      // Reescrito na classe Cachorro
}
```

A herança é declarada explicitamente com o uso de ***:*** após a criação da nova classe, atendendo ao construtor primário da classe pai.

Os métodos sobrescritos recebem a cláusula ***override*** à frente do seu nome, mostrando que essa classe mais especializada tem sua própria versão do método.

Importante ressaltar que ainda é possível chamar a versão do método sobrescrito da classe pai. Para tanto deve-se usar a cláusula ***super*** seguido da chamada ao método, uma vez que super refere-se à classe pai ou superclasse.

Data classes

Kotlin implementa alguns tipos especiais de classes, trazendo comportamentos diferentes funcionais aos programadores. *Data classes* são um exemplo.

Uma ***data class*** é um tipo especial de classe criada para **armazenar informações** de forma prática e automática. Em vez de escrever muito código apenas para guardar e comparar dados, o Kotlin já gera tudo o que é necessário — como toString(), equals(), hashCode() e copy() — de forma automática.

</>

```
data class Produto(val nome: String, val preco: Double)

fun main() {
    val produto1 = Produto("Caneca", 29.9)
    val produto2 = Produto("Caneca", 29.9)
    val produto3 = produto1.copy(preco = 34.9)

    println(produto1)        // Produto(nome=Caneca, preco=29.9)
    println(produto1 == produto2)  // true → comparação pelo conte
    println(produto3)        // Produto(nome=Caneca, preco=34.9)
}
```

Uma ***data class*** serve basicamente para armazenar dados. Ela deve ser usada para entidades simples que não precisem da comportamentos complexos.

> *Singletons em Kotlin*

Singleton é um padrão de projeto (***design pattern***) que permite apenas uma instância de uma classe ao longo de toda a aplicação.

Em Kotlin, a palavra ***object*** é usada para criar um único objeto — ou seja, uma instância única que é criada automaticamente e nunca se repete. Assim sendo, uma classe ***object*** é uma forma simples e rápida para implementar um ***singleton***, o que exige uma boa quantidade de linhas de código para se fazer em outras linguagens.

</>

```
object Configuracao {
    val versao = "1.0"
    val autor = "Maicris"
    fun exibirInfo() {
        println("Versão: $versao - Autor: $autor")
    }
}

fun main() {
    Configuracao.exibirInfo()
}
```

Como pode ser visto anteriormente, para chamar propriedades ou métodos de uma classe ***object*** basta usar seu nome de declaração.

Um ***object*** ainda pode ser declarado dentro de uma classe, definindo uma área estática, em que podem ser declarados propriedades e métodos estáticos da classe. Dessa forma, eles podem ser acessados diretamente pelo nome de declaração da classe, não havendo a necessidade de instanciar objetos para serem acessados. Para obter esse comportamento dentro de uma classe usa-se um ***companion object***.

</>

```
class Usuario(val nome: String) {
    companion object {
        fun criarPadrao(): Usuario {
            return Usuario("Convidado")
        }
    }
}

fun main() {
    val usuario = Usuario.criarPadrao()
    println(usuario.nome)
}
```

Orientação objetos em Kotlin x Java

Veja no quadro a seguir as diferenças entre programação orientada a objeto entre Kotlin e Java (linguagem anterior para desenvolvimento Android).

Conceito	Kotlin	Java
Declaração de classe	Mais concisa. As propriedades podem ser declaradas diretamente no construtor. class Pessoa(val nome: String, var idade: Int)	Precisa declarar os atributos e o construtor separadamente. class Pessoa { String nome; int idade; Pessoa(String n, int i) {...} }
Criação de objetos	Não usa *new*. val p = Pessoa("Ana", 20)	Usa *new* obrigatoriamente. Pessoa p = new Pessoa("Ana", 20);
Herança	As classes são fechadas por padrão. Para permitir herança, usa-se ***open***.	As classes são abertas por padrão, podendo ser herdadas, a menos que sejam ***final***.
Encapsulamento	Criados automaticamente. Podem ser personalizados com ***get()*** e ***set()***.	Precisam ser criados manualmente.
Atributos imutáveis	Usa ***val*** para imutável e ***var*** para mutável.	Precisa usar ***final*** para tornar o atributo imutável.
Classes de dados	data class cria automaticamente ***toString()***, ***equals()***, ***hashCode()*** e ***copy()***.	Precisa sobrescrever manualmente esses métodos.
Métodos e atributos estáticos	Usa ***companion object***.	Usa a palavra-chave ***static***.
Singleton	Simplificado com ***object***.	Precisa implementar manualmente com controle de instância.
Construtores	Pode ter um construtor primário e outros secundários.	Pode ter múltiplos construtores sobrecarregados.
Classes abstratas e interfaces	Suporta herança múltipla de interfaces e permite funções com corpo nelas.	Também suporta, mas métodos em interfaces não podem ter corpo (até o Java 8).

REFLEXÃO

Nesta semana conversamos sobre a criação de **aplicações um pouco mais complexas fazendo uso de interfaces imperativas.** Antes de finalizar seus estudos, propomos que faça uma pausa para pensar sobre alguns pontos. Acompanhe!

1. O que mais me chama atenção na forma como o Kotlin trata herança, classes e objetos?

2. O que muda na sua forma de pensar o código ao usar recursos como *data class* e *object*, em vez de depender de estruturas mais manuais?

3. Ao trabalhar com herança, classes e objetos, o programador precisa abstrair — enxergar padrões, entender o que é comum e o que é diferente. Como esse exercício de abstração pode ajudar em outras áreas da sua vida acadêmica ou profissional?

Sugestões de respostas!

1. O que mais me chama atenção na forma como o Kotlin trata herança, classes e objetos?

O que mais chama atenção é a forma como o Kotlin busca **equilibrar flexibilidade e segurança**. Diferentemente de linguagens mais antigas, o Kotlin incentiva boas práticas ao exigir que classes sejam explicitamente marcadas como open para permitir herança, evitando extensões indevidas por padrão.
Além disso, a sintaxe é mais clara e direta, tornando a definição de classes e objetos mais legível. O Kotlin também integra conceitos modernos da orientação a objetos com recursos funcionais, o que torna o uso de herança mais consciente e menos mecânico, levando o programador a pensar melhor sobre quando realmente faz sentido reutilizar comportamento por herança.

2. O que muda na sua forma de pensar o código ao usar recursos como *data class* e *object*, em vez de depender de estruturas mais manuais?

Ao utilizar recursos como *data class* e *object*, a forma de pensar o código muda significativamente, pois o foco deixa de ser a implementação repetitiva e passa a ser a **modelagem do problema**.
As *data class* permitem representar dados de forma clara e objetiva, sem a necessidade de escrever manualmente métodos como toString, equals ou hashCode. Isso faz com que o programador pense mais no **significado** dos dados do que nos detalhes técnicos.
Já o uso de *object* facilita a criação de instâncias únicas, levando a uma reflexão mais cuidadosa sobre o ciclo de vida dos objetos e a responsabilidade de cada componente no sistema, tornando o código mais organizado e expressivo.

3. Ao trabalhar com herança, classes e objetos, o programador precisa abstrair — enxergar padrões, entender o que é comum e o que é diferente. Como esse exercício de abstração pode ajudar em outras áreas da sua vida acadêmica ou profissional?

O exercício de abstração desenvolvido ao trabalhar com herança, classes e objetos vai além da programação e contribui diretamente para outras áreas da vida acadêmica e profissional.
Ao aprender a identificar padrões, separar o essencial do acessório e organizar ideias de forma lógica, o estudante melhora sua capacidade de **análise, planejamento e resolução de problemas complexos**.
Essa habilidade é útil, por exemplo, na elaboração de projetos, na escrita acadêmica, na gestão de processos e na tomada de decisões no ambiente profissional. Em essência, a abstração ensinada pela programação ajuda a estruturar melhor o pensamento, tornando o indivíduo mais crítico, estratégico e eficiente em diferentes contextos.

Ligando conceitos

O Kotlin vai muito além da sintaxe moderna — ele foi pensado para tornar o código mais seguro, legível e próximo da lógica humana. Ao compreender recursos como o tratamento de valores *nullable*, a criação e o uso de funções, e os princípios da Programação Orientada a Objetos, é possível enxergar o código não apenas como uma sequência de comandos, mas como uma forma de organizar ideias, representar o

mundo e resolver problemas de maneira estruturada. Com ferramentas como data class e object, o Kotlin mostra que é possível unir simplicidade e poder, abrindo caminho para o desenvolvimento de sistemas mais claros, seguros e expressivos.

Referências

DESENVOLVEDORES ANDROID. Disponível em: https://developer.android.com/?hl=pt-br. Acesso em: 24 out. 2025.

JETBRAINS. **Kotlin Programming Language**. Disponível em: https://kotlinlang.org/. Acesso em: Acesso em: 24 out. 2025.


© PUCPR - Todos os direitos reservados.